KAUST

AMCS206 Applied Numerical Methods

Problem Set 2

February 20, 2026

Problem 1

Apply (pencil and paper) naive Gaussian elimination to

$$\begin{cases} 0.003000x_1 + 59.14x_2 = 59.17 \\ 5.291x_1 - 6.130x_2 = 46.78 \end{cases}$$

using 4-digit arithmetic with rounding, and compare the results to the exact solution $x_1 = 10.00$ and $x_2 = 1.000$. Discuss the propagation of the round-off error.
Recall that in 4-digit arithmetic with rounding, the sum of $a = 5/7$ with $b = 1/3$, for example, is done as follows:

$$\begin{aligned} a \oplus b &= fl\left(fl\left(a\right) + fl\left(b\right)\right) \\ &= fl\left(0.7143 \times 10^0 + 0.3333 \times 10^0\right) \\ &= fl\left(1.0476 \times 10^0\right) \\ &= 0.1048 \times 10^1. \end{aligned}$$

Problem 2

Suppose we want to determine the inverse, B, of a nonsingular 3×3 matrix

$$A = \begin{bmatrix} 1 & 2 & -1 \\ 2 & 1 & 0 \\ -1 & 1 & 2 \end{bmatrix}.$$

.

1. First, write the 9 linear equations for the 9 components of B, such that $AB = I$, in which I is the 3×3 identity matrix. Notice that you have 3 similar systems of linear equations in the components of B because the matrix of the coefficients is A and only the right hand side is different.

2. Apply (pencil and paper) forward elimination on the augmented matrix

$$\left[\begin{array}{c:c} A & I \end{array}\right].$$

3. Finally, perform backwards substitution to determine the components of B.

4. What can you conclude about finding the inverse of a matrix?

5. Can you propose another efficient method to determine the inverse of a matrix?

Problem 3

Write a Matlab code that implements Gaussian elimination with scaled partial pivoting. Submit the script. Solve the following linear system of equations, with 3-digit chopping arithmetic,

$$\begin{cases} 3.03x_1 - 12.1x_2 + 14x_3 = -119 \\ -3.03x_1 + 12.1x_2 - 7x_3 = 120 \\ 6.11x_1 - 14.2x_2 + 21x_3 = -139 \end{cases}$$

using your code and submit the output.

Problem 4

Determine the condition number of

$$A = \begin{bmatrix} 1 & 2 \\ 1.0001 & 2 \end{bmatrix},$$

relative to $||\cdot||_\infty$, and decide if we can conclude that a small residual vector implies an accurate approximate solution.

Problem 1

Apply (pencil and paper) naive Gaussian elimination to

$$\begin{cases} 0.003000x_1 + 59.14x_2 = 59.17 \\ 5.291x_1 - 6.130x_2 = 46.78 \end{cases}$$

using 4-digit arithmetic with rounding, and compare the results to the exact solution $x_1 = 10.00$ and $x_2 = 1.000$. Discuss the propagation of the round-off error.
Recall that in 4-digit arithmetic with rounding, the sum of $a = 5/7$ with $b = 1/3$, for example, is done as follows:

$$\begin{aligned} a \oplus b &= fl\left(fl\left(a\right) + fl\left(b\right)\right) \\ &= fl\left(0.7143 \times 10^0 + 0.3333 \times 10^0\right) \\ &= fl\left(1.0476 \times 10^0\right) \\ &= 0.1048 \times 10^1. \end{aligned}$$

$$A = \begin{pmatrix} 0.00300 & 59.14 \\ 5.291 & -6.130 \end{pmatrix} \qquad b = \begin{pmatrix} 59.17 \\ 46.78 \end{pmatrix}$$

$$Ax = b$$

$$(A \mid b) = \left(\begin{array}{cc|c} 0.00300 & 59.14 & 59.17 \\ 5.291 & 6.130 & 46.78 \end{array}\right)$$

$a_{11} = 0.00300 \qquad a_{21} = 5.291$

$a_{11} = 3 \times 10^{-3} \qquad a_{21} = 5291 \cdot 10^{-3}$

$m_{21} = fl(a_{21}/a_{11}) = fl\left(\frac{5291}{3}\right) = fl(1763.6) = 1.764 \times 10^3$

$a_{21}^* = a_{21} - m_{21} \cdot a_{11} = 5.291 - 1.764 \times 10^3 \times 3 \times 10^{-3}$

$= 5.291 \times 10^0 - 1.764 \times 3 \times 10^0 =$

$= 5.291 \times 10^0 - 5292 \times 10^0 = -0.001000 \approx 0$

$a_{22}^* = a_{22} - m_{21} \cdot a_{12} = -6.130 - 5.914 \times 10^1 \times 1.764 \times 10^3$

$fl(1764 \times 59.14) = 104322.96 \approx 1.043 \times 10^5$

$fl(-6.130 - 104300) = -104306.13 \approx -1.043 \times 10^5$

RHS:

$b_{12}^* = fl(b_{12} - fl(m_{12} \times b_{11})) = fl(46.78 - fl(1764 \times 59.17))$

$fl(1764 \times 59.17) = fl(104375.88) \approx 1.044 \times 10^5$

$fl(46.78 - 104400) = fl(-104353.22) = -1.044 \times 10^5$

So we have in row two: $-1.043 \times 10^5 \, x_2 = -1.044 \times 10^5$

$$x_2 = fl\left(\frac{-1.044 \times 10^5}{-1.043 \times 10^5}\right) = fl(1.00095\ldots) = 1.001 \times 10^0$$

$$x_1 = fl\left(\frac{59.17 - fl(59.14 \times 1.001)}{0.003000}\right) =$$

$$= fl\left(\frac{fl(59.17 - 59.20)}{0.003000}\right) = fl(-10.00) = -10.00$$

The actual $x_1 = 10$ so we have 200% relative error.

Since $m_{12} = 1764$ and $a_{11} = 0.003000$

We can tell that multiplying by that factor amplifies the error in the first row.

$$\begin{bmatrix} 1 & 2 & -1 \\ 2 & 1 & 0 \\ -1 & 1 & 2 \end{bmatrix} \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{31} & b_{32} & b_{33} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$\begin{cases} b_{11} + 2b_{21} - b_{31} = 1 \\ b_{12} + 2b_{22} - b_{32} = 0 \\ b_{13} + 2b_{23} - b_{33} = 0 \\ 2b_{11} + 1b_{21} + 0b_{31} = 0 \\ 2b_{12} + 1\cdot b_{22} + 0\cdot b_{32} = 1 \\ 2b_{13} + 1\cdot b_{23} + 0\cdot b_{33} = 0 \\ -1\cdot b_{11} + 1\cdot b_{21} + 2\cdot b_{31} = 0 \\ -1\cdot b_{12} + 1\cdot b_{22} + 2\cdot b_{32} = 0 \\ -1\cdot b_{13} + 1\cdot b_{23} + 2\cdot b_{33} = 1 \end{cases}$$

(1)

(2)

(3)

systems

$$A \begin{pmatrix} b_{11} \\ b_{21} \\ b_{31} \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} \quad (1)$$

$$A \begin{pmatrix} b_{12} \\ b_{22} \\ b_{32} \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} \quad (2)$$

$$A \begin{pmatrix} b_{13} \\ b_{23} \\ b_{33} \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} \quad (3)$$

$$(A|I) = \left(\begin{array}{ccc|ccc} 1 & 2 & -1 & 1 & 0 & 0 \\ 2 & 1 & 0 & 0 & 1 & 0 \\ -1 & 1 & 2 & 0 & 0 & 1 \end{array}\right)$$

Step 2. Forward elimination

$$r_2 \to r_2 - \frac{2}{1} \cdot r_1$$

$$(A|I) \to \left(\begin{array}{ccc|ccc} 1 & 2 & -1 & 1 & 0 & 0 \\ 0 & -3 & 2 & -2 & 1 & 0 \\ -1 & 1 & 2 & 0 & 0 & 1 \end{array}\right)$$

$$r_3 \to r_3 - \frac{-1}{1} r_1 = r_3 + r_1$$

$$\left(\begin{array}{ccc|ccc} 1 & 2 & -1 & 1 & 0 & 0 \\ 0 & -3 & 2 & -2 & 1 & 0 \\ 0 & 3 & 1 & 1 & 0 & 1 \end{array}\right)$$

$$r_3 \to r_3 - \frac{3}{-3} r_2 = r_3 + r_2$$

$$\left(\begin{array}{ccc|ccc} 1 & 2 & -1 & 1 & 0 & 0 \\ 0 & -3 & 2 & -2 & 1 & 0 \\ 0 & 0 & 3 & -1 & 1 & 1 \end{array}\right)$$

Step 3. Backward substitution:

$$r_3 \to r_3 / 3$$

$$\left(\begin{array}{ccc|ccc} 1 & 2 & -1 & 1 & 0 & 0 \\ 0 & -3 & 2 & -2 & 1 & 0 \\ 0 & 0 & 1 & -1/3 & 1/3 & 1/3 \end{array}\right)$$

$r_2 \to r_2/-3$

$$\left(\begin{array}{ccc|ccc} 1 & 2 & -1 & 1 & 0 & 0 \\ 0 & 1 & -2/3 & +2/3 & -1/3 & 0 \\ 0 & 0 & 1 & -1/3 & 1/3 & 1/3 \end{array}\right)$$

$$r_2 \to r_2 - \frac{-2/3}{1} \cdot r_3 = r_2 + \frac{2}{3} r_3$$

$$\left(\begin{array}{ccc|ccc} 1 & 2 & -1 & 1 & 0 & 0 \\ 0 & 1 & 0 & \frac{4}{9} & -1/9 & 2/9 \\ 0 & 0 & 1 & -1/3 & 1/3 & 1/3 \end{array}\right)$$

$$r_1 \to r_1 - \frac{2}{1} \cdot r_2 = r_1 - 2 r_2$$

$$\left(\begin{array}{ccc|ccc} 1 & 0 & -1 & \frac{1}{9} & \frac{2}{9} & \frac{-4}{9} \\ 0 & 1 & 0 & \frac{4}{9} & -1/9 & 2/9 \\ 0 & 0 & 1 & -1/3 & 1/3 & 1/3 \end{array}\right)$$

$$r_1 \to r_1 - \frac{-1}{1} r_3 = r_1 + r_3$$

$$\left(\begin{array}{ccc|ccc} 1 & 0 & 0 & -2/9 & 5/9 & -1/9 \\ 0 & 1 & 0 & \frac{4}{9} & -1/9 & 2/9 \end{array}\right)$$

$$\left(0 \quad 0 \quad 1 \;\middle|\; -\tfrac{1}{3} \quad \tfrac{1}{3} \quad \tfrac{1}{3}\right)$$

Check

$$B = \frac{1}{9}\begin{pmatrix} -2 & 5 & -1 \\ 4 & -1 & 2 \\ -3 & 3 & 3 \end{pmatrix}$$

$$AB = \begin{pmatrix} 1 & 2 & -1 \\ 2 & 1 & 0 \\ -1 & 1 & 2 \end{pmatrix} \frac{1}{9}\begin{pmatrix} -2 & 5 & -1 \\ 4 & -1 & 2 \\ -3 & 3 & 3 \end{pmatrix}$$

$$= \frac{1}{9}\begin{pmatrix} 9 & 0 & 0 \\ 0 & 9 & 0 \\ 0 & 0 & 9 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Conclusion: Finding A^{-1} is equivalent to solving 3 linear systems (the ones from part 1)

$Ax = e_1$, $Ax = e_2$, $Ax = e_3$

Anothe approach of finding it is by using LU decompsition. We decompose it once and solve for n different RHS. Since we need to decompose once we have $\sim O(n^3)$ and for solving the linear systems it will take $n \cdot n^2$. This will give us $O(n^3)$

Problem 4

Determine the condition number of

$$A = \begin{bmatrix} 1 & 2 \\ 1.0001 & 2 \end{bmatrix},$$

relative to $||\cdot||_\infty$, and decide if we can conclude that a small residual vector implies an accurate approximate solution.

$$\kappa(A) = ||A||_\infty ||A^{-1}||_\infty$$

$$||A||_\infty = \max(1+2,\ 1.0001+2) = 3.0001$$

$$A^{-1} = \frac{1}{2-2.0002} \cdot \begin{bmatrix} 2 & -2 \\ -1.0001 & 1 \end{bmatrix} = -\frac{1}{0.0002} \cdot \begin{bmatrix} 2 & -2 \\ -1.0001 & 1 \end{bmatrix}$$

$$= -\frac{10000}{2} \cdot \begin{bmatrix} 2 & -2 \\ -1.0001 & 1 \end{bmatrix} = -5000 \begin{bmatrix} 2 & -2 \\ -1.0001 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} -10000 & 10000 \\ 5000.5 & -5000 \end{bmatrix}$$

$$||A^{-1}||_\infty = 20000 \quad (\text{max abs sum of a row})$$

$$\kappa_\infty(A) = 3.0001 \times 20000 = 60002$$

since $\kappa_\infty(A)$ is moderate between 10^0 and 10^6. Then we can conclude that a small residual means a small error. But due to rule of thumb we can lose about 5 digits of precision. It does result in some loss of accuracy.

Problem 3

SPP.py 1

SPP.py > ...

```python
import numpy as np

def chop(x, t=3):
    if x == 0:
        return 0.0

    n = int(np.floor(np.log10(abs(x))))
    mantissa = x / (10**n)

    shift = 10**(t - 1)
    chopped_mantissa = np.trunc(mantissa * shift) / shift

    return float(chopped_mantissa * (10**n))

def SPP_solver(A, b, t=3):
    n = len(b)

    A_b = np.zeros((n, n + 1))
    for i in range(n):
        for j in range(n):
            A_b[i, j] = chop(A[i, j], t)
        A_b[i, n] = chop(b[i], t)

    scale = np.array([max(abs(val) for val in row[:-1]) for row in A_b])

    for col in range(n - 1):
        max_ratio = -1.0
        pivot_row = col
        for i in range(col, n):
            ratio = chop(abs(A_b[i, col]) / scale[i], t)
            if ratio > max_ratio:
                max_ratio = ratio
                pivot_row = i

        if pivot_row != col:
            A_b[[col, pivot_row]] = A_b[[pivot_row, col]]
            scale[[col, pivot_row]] = scale[[pivot_row, col]]

        for i in range(col + 1, n):
            m = chop(A_b[i, col] / A_b[col, col], t)
            for j in range(col, n + 1):
                product = chop(m * A_b[col, j], t)
                A_b[i, j] = chop(A_b[i, j] - product, t)

    # Back Substitution
    x = np.zeros(n)
    for i in range(n - 1, -1, -1):
        sum_ax = 0.0
        for j in range(i + 1, n):
            product = chop(A_b[i, j] * x[j], t)
            sum_ax = chop(sum_ax + product, t)

        numerator = chop(A_b[i, n] - sum_ax, t)
        x[i] = chop(numerator / A_b[i, i], t)

    return x

A = np.array([[ 3.03, -12.1, 14.0],
              [-3.03,  12.1, -7.0],
              [ 6.11, -14.2, 21.0]], dtype=float)

b = np.array([-119.0, 120.0, -139.0], dtype=float)

sol = SPP_solver(A, b, t=3)
x_exact = np.linalg.solve(A, b)

print(f"Chopped Solution (3-digit): {sol}")
print(f"Exact Solution (Standard):  {x_exact}")

abs_error = np.linalg.norm(sol - x_exact, np.inf)
rel_error = abs_error / np.linalg.norm(x_exact, np.inf)

print(f"Absolute Error (Infinity Norm): {abs_error:.4f}")
print(f"Relative Error (Infinity Norm): {rel_error:.4f}")
```

PROBLEMS 1 OUTPUT PORTS TERMINAL

```
(anm_env) yorovg@Kl-29058 hw2 % python SPP.py
Chopped Solution (3-digit): [-0.163  9.98   0.143]
Exact Solution (Standard):  [2.96544022e-15 1.00000000e+01 1.42857143e-01]
Absolute Error (Infinity Norm): 0.1630
Relative Error (Infinity Norm): 0.0163
(anm_env) yorovg@Kl-29058 hw2 %
```

0 1